Java ATM Application

Technical report to management

Course	Software Design 1 (SDN150S), Project No. 2, Group 2
Prepared for	Management, Faculty of Engineering and the Built Environment, CPUT (Examiner: Mr S.O. Ekwe)
Prepared by	Aubrey Magolego, Mabandla Nkosi, Vhukhudo Ravele, Muvhofhololi Tsanwani (team leader), Athenkosi Khau, Kgonyane Makgoale, Ndzalama Mthombeni, Siyolise Mhlauli
Date	8 November 2026
Source code	GitHub repository atm-project-group2
Word count	1647 words (Executive summary to Conclusion, including tables and captions; excluding this block, references and appendices)

Executive summary

This report explains how Group 2 designed, built and tested a Java desktop ATM for the SDN150S project brief. A customer logs in with an account number and PIN, then withdraws cash, deposits money, checks the balance or exits. We split the program into three layers, agreed every method name in advance and gave each of the eight members their own files. The finished program passes 105 automated backend checks and 34 scripted screen checks, and it caught all eight faults we deliberately injected. We recommend that a future version stores data permanently, uses exact decimal money types and runs its tests automatically.

1 Introduction

ATMs let people withdraw cash, deposit money and check balances without visiting a branch, and they make formal financial services easier to reach in rural areas (CPUT, 2026). The brief asks our group to build a Java program that illustrates one. An authenticated user chooses Withdraw, Deposit, Check Balance or Exit, and the account starts with R10 000.00.

The program must use constructors, access modifiers, control statements and Java frame forms, together with input validation, error handling, readable comments and Git version control (CPUT, 2026). This report is written for management. It explains our approach, presents the evidence that the program works and states the limits of what we built. Section 2 lists our questions, Section 3 discusses the approach, Section 4 gives the results and Section 5 concludes. The pseudocode, UML diagrams and key source code are in Appendices B to D.

2 Sub-questions

Five questions guided the work. Sections 3 and 4 answer them.

SQ1 Structure. How should the program be organised so that eight people can build it in parallel without editing the same file?

SQ2 Correctness and safety. How can the program keep a customer's money and PIN correct and safe?

SQ3 Usability. How can the screens be consistent, clear and hard to misuse?

SQ4 Verification. How do we know the program works, including when users make mistakes?

SQ5 Tools. Which tools did we use, and what are their limits?

3 Discussion

3.1 Software design approach (SQ1, SQ2)

We used object-oriented design with three layers (Figure 1), following the principle of separation of concerns (Sommerville, 2016). The screens only draw and read input. The ATM class applies the rules: login, a card block after three wrong PINs, whole-note withdrawals and a R5 000 daily limit. The Account class owns the data: balance, PIN and history. Because the screens contain no banking rules, a rule can change in one place, and the rules can be tested without opening a window. Every screen receives the same ATM object through its constructor, so all screens show the same balance.

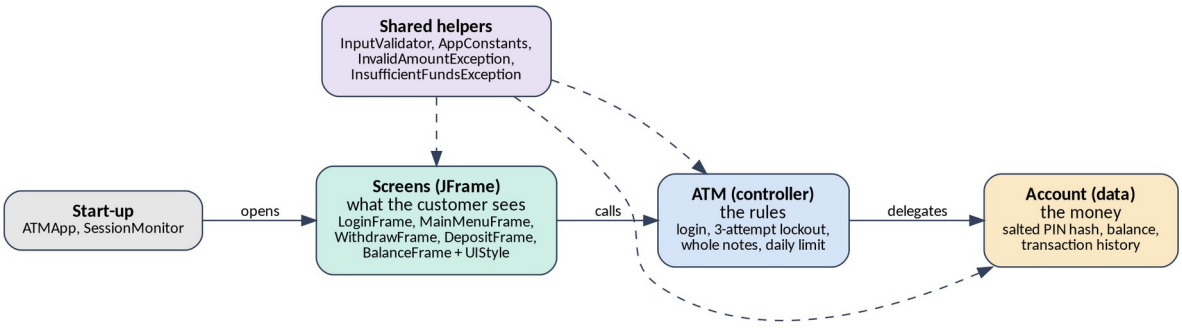


Figure 1: Three-layer structure of the program. Source: Group 2 (own work).

Three further choices follow. First, checked exceptions (InvalidAmountException and InsufficientFundsException) force each screen to handle expected failures, which Bloch (2018) recommends for recoverable conditions. Second, fields are private and classes expose small public methods, so an object cannot be left in an invalid state. Third, fixed values live in AppConstants instead of being scattered through the code as magic numbers (Martin, 2009). The design is recorded in UML (Fowler, 2004) in Appendix C.

3.2 Engineering method (SQ1)

The team agreed a contract-first, iterative method. A shared contract fixed the class names, method signatures and component prefixes (for example txtAmount and btnConfirm) that everyone codes against. Each member then owned whole files, so no two people edited the same file. Members 1 to

3 wrote backend classes without Swing code, and members 4 to 8 built one screen each. Everyone worked on a feature branch, committed small and often, and opened a pull request that the integration lead reviewed and merged (Chacon and Straub, 2014). Compiling stubs of Account, ATM and UIStyle were shared first, so every screen could be built on its own. The timeline set four milestones: stubs (25 October), working screens (1 November), merge and run-through (4 November), and report and video (6 November).

3.3 Implementation (SQ2, SQ3)

Table 1 shows where each required Java concept is used.

Table 1: Required Java concepts and where they appear.

Concept	Where it is used
Constructors	Every class: Account(number, pin, balance), ATM() builds the demo account, each screen receives the shared ATM, Transaction has two overloaded constructors.
Modifiers	private fields, public methods, final for values that never change, static helpers, private constructors on UIStyle, InputValidator and AppConstants.
Control statements	if / else chains (login checks, greeting), switch (menu buttons), for loops (statement, daily total), try / catch / finally (withdraw and deposit).
Java frame forms	Five hand-coded JFrame screens. UIStyle supplies shared colours, fonts and custom-painted rounded buttons.
Validation and errors	InputValidator accepts only plain amounts; two exceptions carry customer-friendly messages shown in red or green.

We added five features beyond the brief. The PIN is stored only as a salted PBKDF2 hash, a slow one-way function, never as text. Amounts are rounded to whole cents after every operation, because binary floating point cannot store most decimals exactly (Goldberg, 1991). A transaction history feeds a mini statement and the daily limit. An inactivity monitor signs the customer out after 120 seconds. Quick-amount buttons speed up common transactions. Appendix D contains the core source code, and Figure 5 shows the withdrawal logic.

3.4 Skills and tools (SQ5)

The project needed object-oriented programming, event-driven GUI programming, defensive validation with regular expressions, test writing, Git collaboration and technical writing. Table 2 lists the tools and, as the marking guide asks, their limits and assumptions.

Table 2: Tools, their uses and their limitations.

Tool	Used for	Limitation or assumption
Java and Swing	Screens, no external libraries	Swing is an older toolkit; its official tutorial was written for JDK 8 (Oracle, n.d.). Look and feel differs by operating system, so we fixed the cross-platform look. Tested on OpenJDK 21.
VS Code with Java extensions	Editing, running, debugging	No drag-and-drop GUI designer, so every layout is hand-coded (Microsoft, n.d.).
Git and GitHub	Version control, pull requests	Works only if members commit often and keep to their own files.
double for money	Agreed team money type	Inexact decimals. We round to cents; production code should use BigDecimal or whole cents.
PBKDF2 PIN hash	Hide the PIN in memory	A four-digit PIN has only 10 000 possibilities, so any hash can be guessed offline. The real protection is the three-attempt block. Real systems use slow, modern algorithms with higher work factors (OWASP, n.d.).
BackendTests and GuiWalkthrough	Repeatable testing	Hand-written, not JUnit. The screen test needs a display, checks messages rather than pixels and relies on short waits.
Graphviz	UML drawing	Diagrams are generated from text and must be updated when the code changes.

4 Results and analysis

4.1 Backend tests (SQ4)

BackendTests runs 105 checks without any library. All passed (Table 3) in about three seconds, because the backend needs no windows.

Table 3: Backend test results.

Test group	Checks	Passed
Account: constructor, PIN, deposit, withdraw, history	27	27
Transaction	3	3
InputValidator: accepted and rejected input	31	31
InputValidator: error messages	6	6
ATM: login and card block	14	14
ATM: deposit, withdraw, statement	11	11
ATM: whole notes and daily limit	11	11
End to end: scenario from the brief	2	2
Total	**105**	**105**

4.2 Can the tests find faults? (SQ4)

We injected eight deliberate faults, one at a time, for example allowing an overdraft at exactly the balance, a lockout that is off by one, and an ignored daily limit. All eight were detected. An earlier

run missed one: removing the negative-number branch still rejected -50, only with a vaguer message, because messages were untested. We added six message checks and that fault is now caught.

4.3 Screen tests (SQ3, SQ4)

GuiWalkthrough types into the real screens, clicks the buttons and checks the messages. All 34 checks passed. They cover login errors, the card block (the program exits with System.exit(0)), every withdraw and deposit error, Back navigation, the statement and the inactivity sign-out. Figure 2 shows three captured screens. The screenshots exposed layout defects that unit tests could not, such as a cut-off R1000 button and a success message that overflowed its label. We fixed them and re-ran every test.

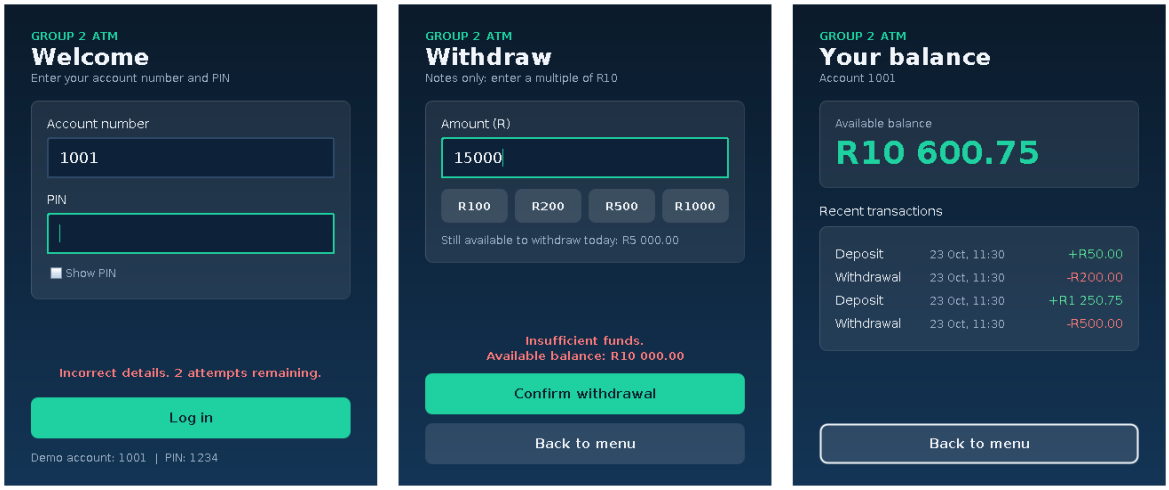


Figure 2: Captured screens: (a) login error, (b) insufficient funds, (c) balance with mini statement. Source: Group 2 (own work).

4.4 Requirements and analysis

Table 4: Brief requirements and results.

Requirement in the brief	Result
Authenticated user, starting balance R10 000.00	Met. Login by account number and PIN; demo account 1001.
Withdraw, deposit and check balance with messages	Met. R500 withdrawn gives R9 500.00; statement added.
Exit using System.exit(0)	Met. Exit asks for confirmation first.
Constructors, modifiers, control statements, frame forms	Met (Table 1).
Error handling and input validation	Met. 37 validation and error checks pass.

Three decisions worked well. Separating rules from screens made the rules testable in seconds. Checking funds before the daily limit gives the clearest message when a customer asks for more than the balance. Rounding to cents removed the 0.10 + 0.20 error. The weaknesses are that data lives

only in memory, so balances reset when the program closes, and that there is a single demo account.

5 Conclusion

Group 2 delivered a complete Java ATM that meets every requirement in the brief. A layered design, a shared contract and file ownership let eight people work in parallel. Verification at two levels, 105 backend checks and 34 screen checks, gives confidence in the result, and fault injection showed that the tests can detect real bugs.

We recommend four improvements: store accounts in a file or database so balances persist; use BigDecimal or whole cents for money; replace the hand-written runners with JUnit and run them on every pull request; and add accessibility features such as larger text and screen-reader labels.

References

Bloch, J. (2018) *Effective Java*. 3rd edn. Boston: Addison-Wesley.

Cape Peninsula University of Technology (CPUT) (2026) *Software Design 1 (SDN150S) Project Brief*. Cape Town: CPUT, Faculty of Engineering and the Built Environment.

Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. New York: Apress. Available at: https://git-scm.com/book (Accessed: 23 October 2026).

Fowler, M. (2004) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston: Addison-Wesley.

Goldberg, D. (1991) 'What every computer scientist should know about floating-point arithmetic', *ACM Computing Surveys*, 23(1), pp. 5-48.

Martin, R.C. (2009) *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall.

Microsoft (n.d.) *Java in Visual Studio Code*. Available at: https://code.visualstudio.com/docs/languages/java (Accessed: 23 October 2026).

Oracle (n.d.) *The Java Tutorials: Creating a GUI with Swing*. Available at: https://docs.oracle.com/javase/tutorial/uiswing/ (Accessed: 23 October 2026).

OWASP (n.d.) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 23 October 2026).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

Appendix A: Team roles and file ownership

Member	Role	Files	Report part
Aubrey Magolego	Data model	Account.java, AppConstants.java, Transaction.java	UML diagram
Mabandla Nkosi	Controller	ATM.java	Design approach, engineering method
Vhukhudo Ravele	Validation	InputValidator.java, InvalidAmountException.java, InsufficientFundsException.java	Pseudocode
Muvhofhololi Tsanwani	Team leader, login screen	LoginFrame.java, UIStyle.java	Introduction, sub-questions
Athenkosi Khau	Menu screen	MainMenuFrame.java	Tools and skills
Kgonyane Makgoale	Withdraw screen, testing	WithdrawFrame.java, test/BackendTests.java, test/GuiWalkthrough.java	Results and analysis
Ndzalama Mthombeni	Deposit screen, testing	DepositFrame.java, test/BackendTests.java, test/GuiWalkthrough.java	Results and analysis
Siyolise Mhlauli	Integration lead	ATMApp.java, BalanceFrame.java, SessionMonitor.java, README.md	Conclusion, references

Appendix B: Pseudocode

Written in plain language, without programming syntax, and matching the implemented code.

Program start (ATMApp)

```
START the program
    Choose the cross-platform look so colours are identical on every computer
    On the screen-drawing thread
        Create one ATM object, which sets up the demo account
        Show the Login screen, giving it the ATM object
        Start the inactivity monitor with the 120-second limit
END
```

Account

```
WHEN a new Account is created
    Receive an account number, a PIN and a starting balance
    IF the account number is blank, the PIN is not four digits, or the balance is negative
        Refuse to create the account
    Make a random salt
    Mix the salt with the PIN and scramble it into a hash
    Store the account number, salt, hash and balance (rounded to cents)
    Start with an empty history

TO check a PIN
    IF no PIN was given
        Report FALSE
    Scramble the given PIN with the stored salt
    IF the result matches the stored hash
        Report TRUE
    ELSE
        Report FALSE

TO deposit an amount
    Round the amount to cents
    IF the amount is not a positive number
        Refuse the deposit
    Add the amount to the balance
    Record a "deposit" entry: amount, new balance and the time

TO withdraw an amount
    Round the amount to cents
    IF the amount is not a positive number
        Refuse the withdrawal
    IF the amount is more than the balance
        Raise an "insufficient funds" error holding the balance and the amount
    Subtract the amount from the balance
    Record a "withdrawal" entry: amount, new balance and the time

TO total the withdrawals made on a given day
    Set the total to zero
    FOR EACH entry in the history
        IF the entry is a withdrawal made on that day
            Add its amount to the total
    Report the total
```

ATM (controller)

```
WHEN the ATM is created
    Create the demo account (1001, PIN 1234, R10 000.00) and add it to the account list
    Set the current account to nothing
    Set failed attempts to zero

TO authenticate a user (account number and PIN given)
    IF the card is already blocked
        Report FALSE
    Search the account list for the account number
    IF an account was found AND its PIN matches
        Set the current account to the account found
        Set failed attempts to zero
        Report TRUE
    ELSE
```

```
        Add one to failed attempts
        Report FALSE

TO check whether the card is blocked
    Report TRUE when failed attempts has reached 3

TO withdraw an amount
    Make sure a customer is logged in
    IF the amount is not a whole number of R10 notes
        Raise an "invalid amount" error
    IF the amount is more than the balance
        Raise an "insufficient funds" error
    IF the amount is more than what is left of today's R5 000 limit
        Raise an "invalid amount" error about the daily limit
    Ask the current account to withdraw the amount

TO work out the remaining daily limit
    Ask the current account how much was withdrawn today
    Subtract that from R5 000
    Report the result, never less than zero

TO deposit an amount
    Make sure a customer is logged in
    Ask the current account to deposit the amount

TO get the balance
    Make sure a customer is logged in
    Ask the current account for its balance and report it

TO list the most recent transactions (a count is given)
    Make sure a customer is logged in
    Start with an empty list
    Walk through the history from newest to oldest
        Add each entry until the count is reached or the history ends
    Report the list

TO log out
    Set the current account to nothing
```

InputValidator

```
TO convert text to an amount
    Remove all spaces and one leading "R" from the text
    IF the text is empty
        Raise an error: please enter an amount
    ELSE IF the text starts with a minus sign
        Raise an error: an amount cannot be negative
    ELSE IF the text has more than two decimal places
        Raise an error: use at most two decimal places
    ELSE IF the text has more than nine digits before the decimal point
        Raise an error: that amount is too large
    ELSE IF the text is not digits with at most one decimal point
        Raise an error: use digits only
    Convert the text to a number
    IF the number is zero
        Raise an error: the amount must be greater than zero
    Report the number

TO check a PIN
    IF the PIN is exactly four digits
        Report TRUE
    ELSE
        Report FALSE

TO check an account number
    IF the account number is four to ten digits
        Report TRUE
    ELSE
        Report FALSE
```

Login screen

```
WHEN the Login screen is created
    Receive the ATM object and keep it
    Draw the account number field, the PIN field (characters hidden),
```

```
        a "Show PIN" tick box, the Login button and a message line
    Make the Enter key press the Login button

WHEN the user ticks or unticks "Show PIN"
    IF it is ticked
        Show the PIN digits
    ELSE
        Hide the PIN digits behind dots

WHEN the user presses Login
    Read the account number and the PIN from the fields
    IF either field is empty
        Show a message asking for both details
    ELSE IF the account number is not digits only
        Show a message that account numbers contain digits only
    ELSE IF the PIN is not four digits
        Show a message that the PIN must be four digits
    ELSE IF the ATM accepts the details
        Close the Login screen
        Open the Main Menu screen
    ELSE IF the card is now blocked
        Show a "card blocked" message
        Exit the program
    ELSE
        Show an "incorrect details" message with the attempts remaining
        Clear the PIN field
        Wobble the window left and right
```

Main Menu screen

```
WHEN the Main Menu is created
    Receive the ATM object and keep it
    IF the hour is before 12
        Greet with "Good morning"
    ELSE IF the hour is before 18
        Greet with "Good afternoon"
    ELSE
        Greet with "Good evening"
    Draw four buttons: Withdraw, Deposit, Check Balance, Exit

WHEN the user presses a button
    CASE Withdraw
        Close the menu and open the Withdraw screen
    CASE Deposit
        Close the menu and open the Deposit screen
    CASE Check Balance
        Close the menu and open the Balance screen
    CASE Exit
        Ask "End your session and exit the ATM?"
        IF the user answers yes
            Ask the ATM to log out
            Close the whole program
```

Withdraw screen

```
WHEN the Withdraw screen is created
    Receive the ATM object and keep it
    Draw the amount field, the quick-amount buttons, a label for the
        remaining daily limit, a Confirm button, a Back button and a message line
    Show how much may still be withdrawn today

WHEN the user presses a quick-amount button
    Put that amount into the amount field

WHEN the user presses Confirm
    TRY
        Convert the text in the amount field to a number
        Ask the ATM to withdraw that amount
        Show a green "withdrawal successful" message with the new balance
        Clear the amount field
    IF the amount was invalid, not whole notes, or over the daily limit
        Show the reason in red
    IF there were insufficient funds
        Show a red "insufficient funds" message with the available balance
    ALWAYS
```

```
        Refresh the remaining daily limit label
        Put the cursor back in the amount field

WHEN the user presses Back
    Close this screen and open the Main Menu
```

Deposit screen

```
WHEN the Deposit screen is created
    Receive the ATM object and keep it
    Draw the amount field, the quick-amount buttons, a label for the current
        balance, a Confirm button, a Back button and a message line
    Show the current balance

WHEN the user presses a quick-amount button
    Put that amount into the amount field

WHEN the user presses Confirm
    TRY
        Convert the text in the amount field to a number
        Ask the ATM to deposit that amount
        Show a green "deposit successful" message with the new balance
        Clear the amount field
    IF the amount was invalid
        Show the reason in red
    ALWAYS
        Refresh the current balance label
        Put the cursor back in the amount field

WHEN the user presses Back
    Close this screen and open the Main Menu
```

Balance screen

```
WHEN the Balance screen is created
    Receive the ATM object and keep it
    Ask the ATM for the balance and show it large, formatted as rand with two decimals
    Ask the ATM for the 5 most recent transactions
    IF there are none
        Show "No transactions yet"
    ELSE
        FOR EACH transaction, newest first
            Draw one line: type, date and time, and the amount
            IF it is a deposit
                Colour the amount green with a plus sign
            ELSE
                Colour the amount red with a minus sign

WHEN the user presses Back
    Close this screen and open the Main Menu
```

Inactivity monitor (SessionMonitor)

```
WHEN the monitor starts
    Start a countdown of 120 seconds
    WHENEVER the mouse moves, a mouse button is pressed or a key is pressed
        Restart the countdown

WHEN the countdown reaches zero
    IF nobody is logged in
        Restart the countdown
    ELSE
        Ask the ATM to log out
        Open a new Login screen
        Close every other window
        Tell the customer they were signed out for their own safety
```

Appendix C: UML diagrams

Notation: hollow triangle = inheritance; diamond = aggregation (open) or composition (filled); dashed arrow = dependency; + public, - private.

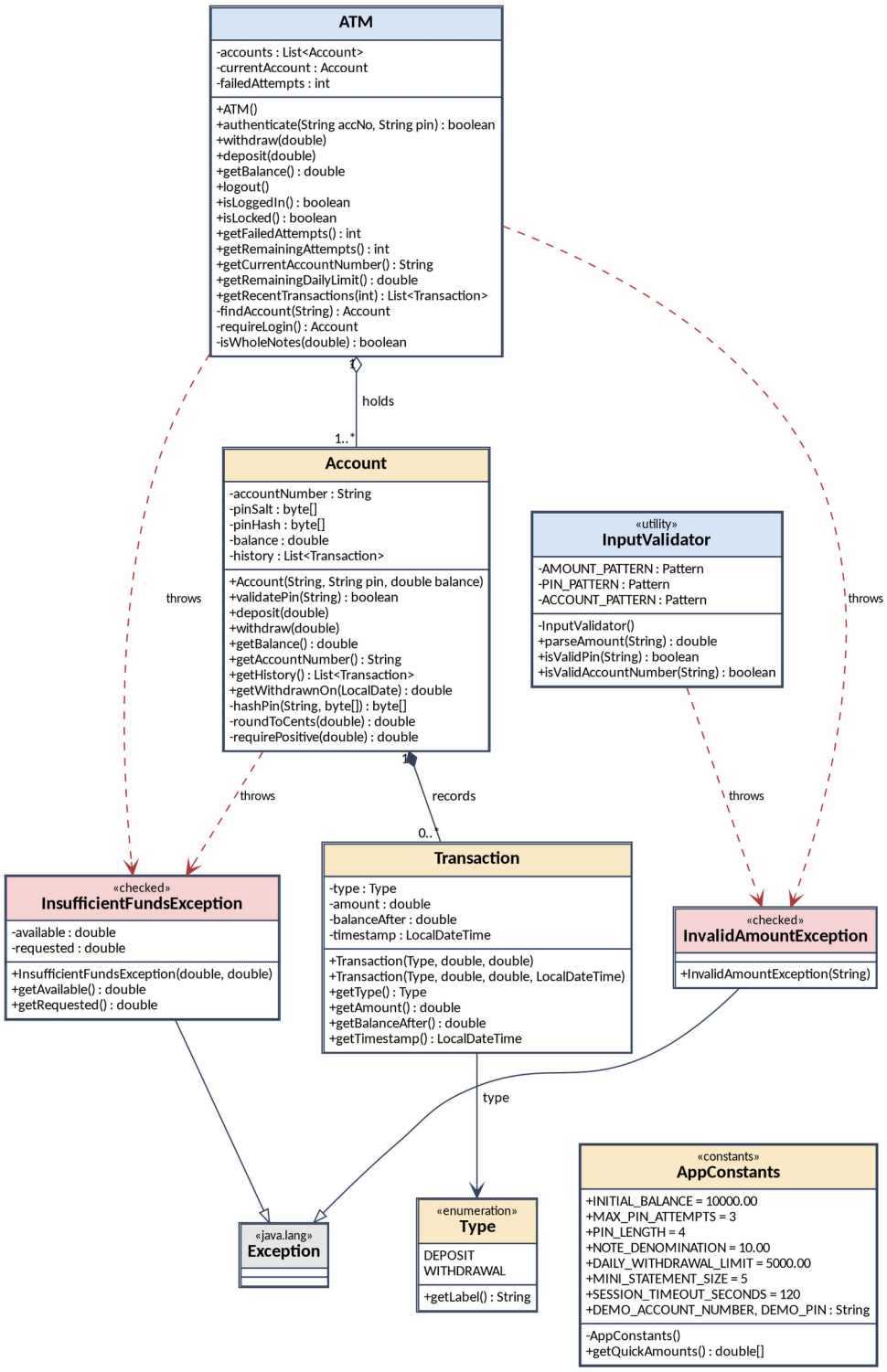


Figure 3: UML class diagram, backend and data model (no Swing code). AppConstants is read by ATM, Account, InputValidator and the screens (arrows omitted for clarity). Source: Group 2 (own work).

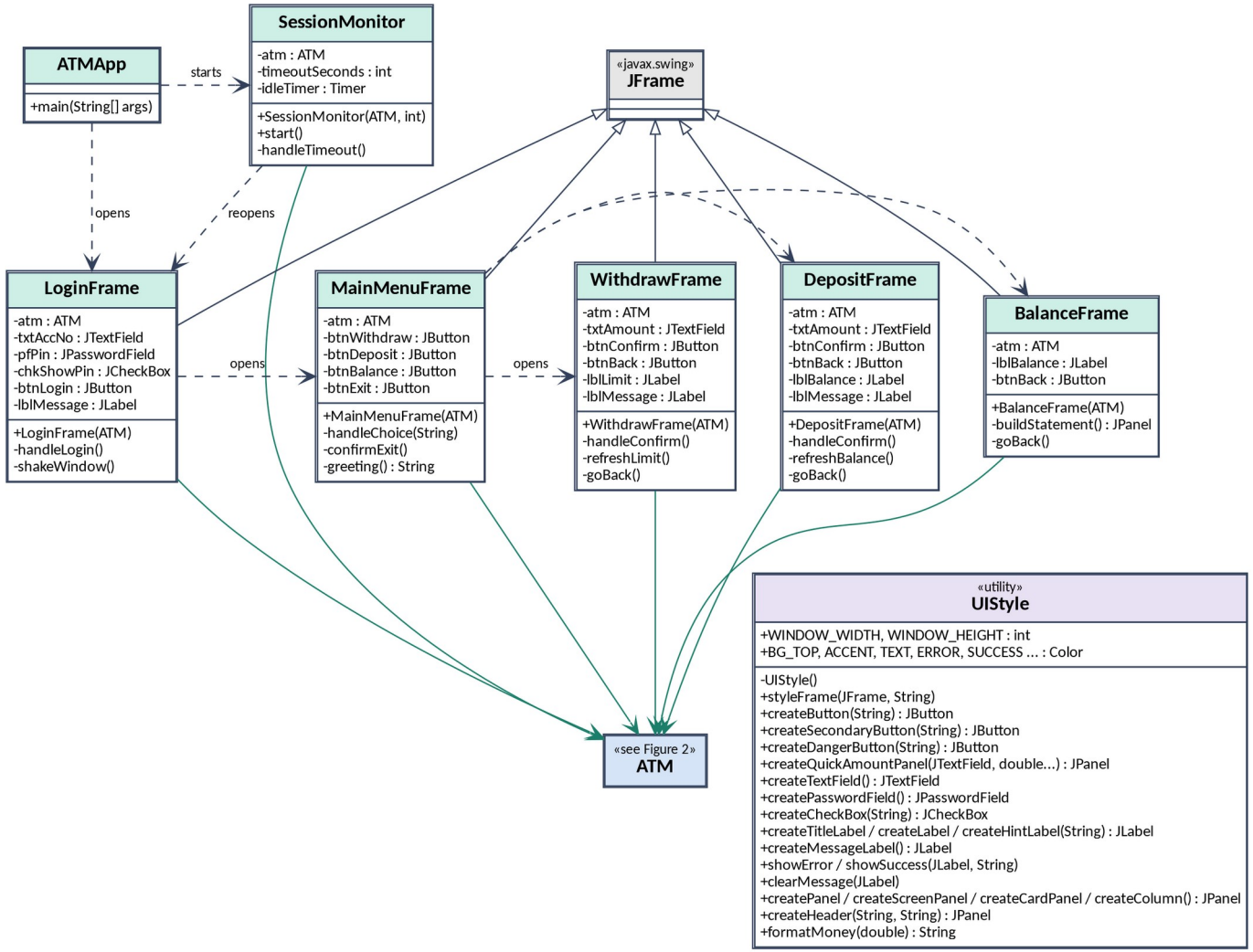


Figure 4: UML class diagram, screens and start-up. ATMApp creates the single ATM object; every screen uses UIStyle for its look (arrows omitted for clarity). Source: Group 2 (own work).

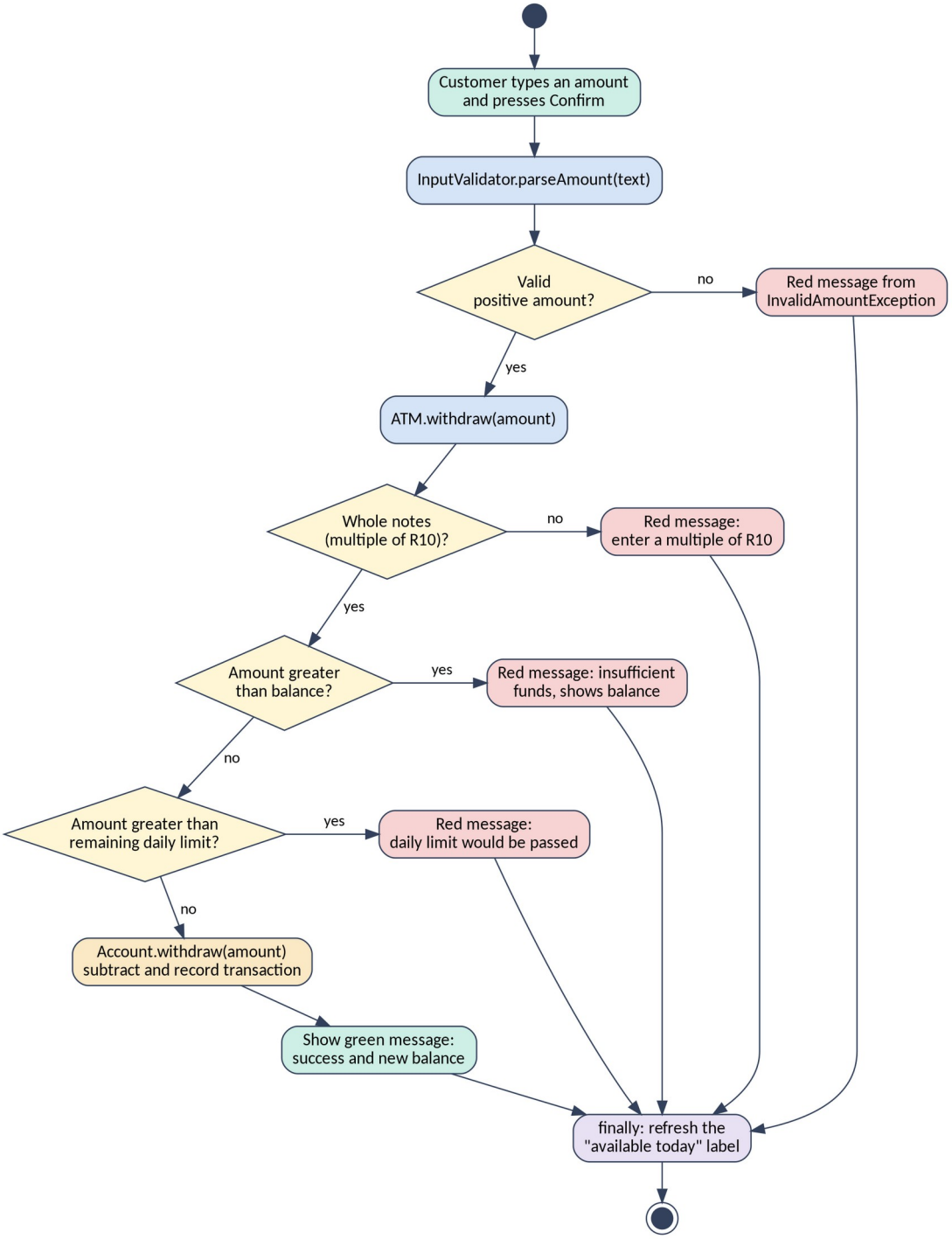


Figure 5: UML activity diagram, withdrawing cash. Source: Group 2 (own work).

Appendix D: Selected source code

The core logic classes and one complete screen. The other ten source files are in the repository (src/atm).

D.1 Account.java (Aubrey Magolego)

```
/*
 * File:        Account.java
 * Author:      Aubrey Magolego
 * Role:        Member 1, Data model
 * Group:       Group 2, SDN150S (Software Design 1), CPUT
 * Description: One bank account: its number, PIN, balance and history.
 *              The PIN is never stored as text. Only a salted PBKDF2 hash
 *              (a deliberately slow one-way function) is kept, so the real
 *              PIN cannot be read back from memory.
 *              This class knows nothing about screens or login attempts.
 */
package atm;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

public class Account {

    // One random generator shared by all accounts (used to create salts)
    private static final SecureRandom RANDOM = new SecureRandom();
    private static final int SALT_LENGTH = 16;

    // PIN hashing settings. Slow on purpose: every guess costs an attacker the same time.
    // 120 000 rounds keeps the demo and the tests quick. A real bank would use far more.
    private static final String HASH_ALGORITHM = "PBKDF2WithHmacSHA256";
    private static final int HASH_ROUNDS = 120_000;
    private static final int HASH_BITS = 256;

    // ------------------------------------------------------------------
    // Fields: all private, reached only through the public methods below
    // ------------------------------------------------------------------
    private final String accountNumber;
    private final byte[] pinSalt;              // random bytes mixed into the hash
    private final byte[] pinHash;              // PBKDF2(PIN, salt), never the PIN itself
    private double balance;                    // always kept rounded to cents
    private final List<Transaction> history;   // oldest first

    /**
     * Creates an account.
     *
     * @param accountNumber the account number, for example "1001"
     * @param pin           exactly four digits
     * @param balance       opening balance in rand, zero or more
     * @throws IllegalArgumentException if any argument breaks those rules
     */
    public Account(String accountNumber, String pin, double balance) {
        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException("An account number is required.");
        }
        if (!InputValidator.isValidPin(pin)) {
            throw new IllegalArgumentException(
                    "A PIN must be exactly " + AppConstants.PIN_LENGTH + " digits.");
        }
        if (Double.isNaN(balance) || Double.isInfinite(balance) || balance < 0) {
            throw new IllegalArgumentException("The opening balance cannot be negative.");
        }

        this.accountNumber = accountNumber;
        this.pinSalt = new byte[SALT_LENGTH];
        RANDOM.nextBytes(this.pinSalt);
        this.pinHash = hashPin(pin, this.pinSalt);
        this.balance = roundToCents(balance);
        this.history = new ArrayList<>();
    }

    // ------------------------------------------------------------------
    // PIN check
    // ------------------------------------------------------------------

    /** True when the given PIN matches the stored hash. */
```

```
public boolean validatePin(String pin) {
    // A stored PIN is always four digits, so anything else can never match
    if (!InputValidator.isValidPin(pin)) {
        return false;
    }
    // isEqual compares every byte, so timing does not reveal how close a guess was
    return MessageDigest.isEqual(pinHash, hashPin(pin, pinSalt));
}

// ---------------------------------------------------------------
// Money movement
// ---------------------------------------------------------------

/** Adds money to the balance and records the transaction. */
public void deposit(double amount) {
    double cents = requirePositive(amount);

    balance = roundToCents(balance + cents);
    history.add(new Transaction(Transaction.Type.DEPOSIT, cents, balance));
}

/**
 * Removes money from the balance and records the transaction.
 *
 * @throws InsufficientFundsException if the amount is more than the balance
 */
public void withdraw(double amount) throws InsufficientFundsException {
    double cents = requirePositive(amount);

    if (cents > balance) {
        throw new InsufficientFundsException(balance, cents);
    }

    balance = roundToCents(balance - cents);
    history.add(new Transaction(Transaction.Type.WITHDRAWAL, cents, balance));
}

// ---------------------------------------------------------------
// Read-only information
// ---------------------------------------------------------------

public double getBalance() {
    return balance;
}

public String getAccountNumber() {
    return accountNumber;
}

/** A snapshot of the history, oldest first. Changing it changes nothing here. */
public List<Transaction> getHistory() {
    return List.copyOf(history);
}

/** Total withdrawn on one calendar day (used for the daily limit). */
public double getWithdrawnOn(LocalDate day) {
    double total = 0;
    for (Transaction t : history) {
        boolean isWithdrawal = t.getType() == Transaction.Type.WITHDRAWAL;
        if (isWithdrawal && t.getTimestamp().toLocalDate().equals(day)) {
            total += t.getAmount();
        }
    }
    return roundToCents(total);
}

// ---------------------------------------------------------------
// Private helpers
// ---------------------------------------------------------------

/** Turns a PIN and a salt into a hash using PBKDF2 (built into Java, no libraries needed). */
private static byte[] hashPin(String pin, byte[] salt) {
    PBEKeySpec spec = new PBEKeySpec(pin.toCharArray(), salt, HASH_ROUNDS, HASH_BITS);
    try {
        return SecretKeyFactory.getInstance(HASH_ALGORITHM).generateSecret(spec).getEncoded();
    } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
        // Every Java platform provides PBKDF2, so this cannot normally happen
        throw new IllegalStateException("PIN hashing is not available.", e);
    } finally {
        spec.clearPassword();      // wipe the copy of the PIN held by the spec
    }
}

/** Rounds to whole cents so that repeated sums never drift (0.1 + 0.2 problem). */
private static double roundToCents(double value) {
```

```
        return Math.round(value * 100.0) / 100.0;
    }

    /** Defends the data model even if a screen forgot to validate. */
    private static double requirePositive(double amount) {
        if (Double.isNaN(amount) || Double.isInfinite(amount) || roundToCents(amount) <= 0) {
            throw new IllegalArgumentException("The amount must be a positive number.");
        }
        return roundToCents(amount);
    }
}
```

D.2 ATM.java (Mabandla Nkosi)

```
/*
 * File:        ATM.java
 * Author:      Mabandla Nkosi
 * Role:        Member 2, Controller
 * Group:       Group 2, SDN150S (Software Design 1), CPUT
 * Description: The brain of the machine. It checks logins (and blocks the
 *              card after too many wrong attempts), enforces the cash rules
 *              (whole notes, daily limit) and passes the transactions on to
 *              the Account. Screens only call this class, they contain no
 *              banking rules of their own.
 */
package atm;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class ATM {

    // ------------------------------------------------------------------
    // State: private, so no screen can change it behind our back
    // ------------------------------------------------------------------
    private final List<Account> accounts;
    private Account currentAccount;      // null while nobody is logged in
    private int failedAttempts;          // wrong logins since the last success

    /** Creates the machine and its demo account (1001, PIN 1234, R10 000.00). */
    public ATM() {
        accounts = new ArrayList<>();
        accounts.add(new Account(AppConstants.DEMO_ACCOUNT_NUMBER,
                AppConstants.DEMO_PIN, AppConstants.INITIAL_BALANCE));

        currentAccount = null;
        failedAttempts = 0;
    }

    // ------------------------------------------------------------------
    // Login and logout
    // ------------------------------------------------------------------

    /**
     * Checks an account number and PIN.
     * A wrong account and a wrong PIN look identical to the caller, so
     * an attacker cannot learn which account numbers exist.
     *
     * @return true if the login succeeded; false if it failed or the card is blocked
     */
    public boolean authenticate(String accNo, String pin) {
        if (isLocked()) {
            return false;          // blocked cards get no more tries
        }

        Account found = findAccount(accNo);

        if (found != null && found.validatePin(pin)) {
            currentAccount = found;
            failedAttempts = 0;
            return true;
        }

        failedAttempts++;
        return false;
    }

    /** Ends the session. The balance and history stay on the account. */
    public void logout() {
        currentAccount = null;
    }

    public boolean isLoggedIn() {
        return currentAccount != null;
    }

    /** True once MAX_PIN_ATTEMPTS wrong logins have happened in a row. */
    public boolean isLocked() {
        return failedAttempts >= AppConstants.MAX_PIN_ATTEMPTS;
    }

    public int getFailedAttempts() {
        return failedAttempts;
    }

    public int getRemainingAttempts() {
```

```
    return Math.max(0, AppConstants.MAX_PIN_ATTEMPTS - failedAttempts);
}

// ---------------------------------------------------------------
// Transactions
// ---------------------------------------------------------------

/**
 * Withdraws cash after applying the machine rules in this order:
 * whole notes, enough money in the account, then the daily limit.
 * (Funds are checked before the limit so that asking for more than the
 * balance always gives the clearest message: "insufficient funds".)
 *
 * @throws InvalidAmountException     not a multiple of the note size, or over the daily limit
 * @throws InsufficientFundsException more than the balance
 */
public void withdraw(double amount) throws InsufficientFundsException, InvalidAmountException {
    Account account = requireLogin();

    if (!isWholeNotes(amount)) {
        throw new InvalidAmountException("The ATM dispenses notes only. "
                + "Enter a multiple of R" + (int) AppConstants.NOTE_DENOMINATION + ".");
    }
    if (amount > account.getBalance()) {
        throw new InsufficientFundsException(account.getBalance(), amount);
    }
    if (amount > getRemainingDailyLimit()) {
        throw new InvalidAmountException("This would pass your daily withdrawal limit.");
    }

    account.withdraw(amount);
}

/** Deposits money into the logged-in account. */
public void deposit(double amount) {
    requireLogin().deposit(amount);
}

public double getBalance() {
    return requireLogin().getBalance();
}

public String getCurrentAccountNumber() {
    return requireLogin().getAccountNumber();
}

/** How much more the customer may withdraw today. */
public double getRemainingDailyLimit() {
    double withdrawnToday = requireLogin().getWithdrawnOn(LocalDate.now());
    double remaining = AppConstants.DAILY_WITHDRAWAL_LIMIT - withdrawnToday;
    return Math.max(0, Math.round(remaining * 100.0) / 100.0);
}

/**
 * The latest transactions, newest first.
 *
 * @param count the most entries wanted
 */
public List<Transaction> getRecentTransactions(int count) {
    List<Transaction> all = requireLogin().getHistory();
    List<Transaction> recent = new ArrayList<>();

    // Walk backwards from the newest entry until we have enough
    for (int i = all.size() - 1; i >= 0 && recent.size() < count; i--) {
        recent.add(all.get(i));
    }
    return recent;
}

// ---------------------------------------------------------------
// Private helpers
// ---------------------------------------------------------------

/** Looks an account up by number, or returns null if there is none. */
private Account findAccount(String accNo) {
    for (Account account : accounts) {
        if (account.getAccountNumber().equals(accNo)) {
            return account;
        }
    }
    return null;
}

/** Returns the logged-in account or fails loudly if nobody is logged in. */
private Account requireLogin() {
```

```
        if (currentAccount == null) {
            throw new IllegalStateException("No customer is logged in.");
        }
        return currentAccount;
    }

    /** Works in whole cents, because double arithmetic on rand values is inexact. */
    private boolean isWholeNotes(double amount) {
        long cents = Math.round(amount * 100.0);
        long noteCents = Math.round(AppConstants.NOTE_DENOMINATION * 100.0);
        return cents % noteCents == 0;
    }
}
```

D.3 InputValidator.java (Vhukhudo Ravele)

```
/*
 * File:        InputValidator.java
 * Author:      Vhukhudo Ravele
 * Role:        Member 3, Validation
 * Group:       Group 2, SDN150S (Software Design 1), CPUT
 * Description: Turns raw text typed by the customer into safe values.
 *              Every check is strict: anything that is not a plain positive
 *              amount (letters, symbols, scientific notation, negatives,
 *              more than two decimals) is rejected with a clear message.
 */
package atm;

import java.util.regex.Pattern;

public final class InputValidator {

    // Whole rand with at most 2 decimals, up to 9 digits before the point
    private static final Pattern AMOUNT_PATTERN = Pattern.compile("\\d{1,9}(\\.\\d{1,2})?");

    // Same shape but with 3 or more decimals, only used to pick a better message
    private static final Pattern TOO_MANY_DECIMALS = Pattern.compile("\\d+\\.\\d{3,}");

    // Digits only, 10 or more of them
    private static final Pattern TOO_LARGE = Pattern.compile("\\d{10,}(\\.\\d+)?");

    // A PIN is exactly PIN_LENGTH digits (ASCII digits only)
    private static final Pattern PIN_PATTERN =
            Pattern.compile("\\d{" + AppConstants.PIN_LENGTH + "}");

    // Account numbers are 4 to 10 digits
    private static final Pattern ACCOUNT_PATTERN = Pattern.compile("\\d{4,10}");

    /** Private constructor: all methods are static, nobody needs an object. */
    private InputValidator() {
    }

    /**
     * Converts the text from an amount field into a valid positive amount.
     *
     * @param text what the customer typed (spaces and a leading "R" are allowed)
     * @return the amount as a double
     * @throws InvalidAmountException with a customer-friendly reason if rejected
     */
    public static double parseAmount(String text) throws InvalidAmountException {

        // Step 1: tidy the input ("R 1 500.50" becomes "1500.50")
        String cleaned = (text == null) ? "" : text.replaceAll("\\s+", "");
        if (cleaned.startsWith("R") || cleaned.startsWith("r")) {
            cleaned = cleaned.substring(1);
        }

        // Step 2: reject anything that is not a plain amount, with the best message
        if (cleaned.isEmpty()) {
            throw new InvalidAmountException("Please enter an amount.");
        } else if (cleaned.startsWith("-")) {
            throw new InvalidAmountException("An amount cannot be negative.");
        } else if (TOO_MANY_DECIMALS.matcher(cleaned).matches()) {
            throw new InvalidAmountException("Use at most two decimal places (cents).");
        } else if (TOO_LARGE.matcher(cleaned).matches()) {
            throw new InvalidAmountException("That amount is too large.");
        } else if (!AMOUNT_PATTERN.matcher(cleaned).matches()) {
            throw new InvalidAmountException("Use digits only, with an optional decimal point.");
        }

        // Step 3: safe to convert now, the pattern guarantees a valid number
        double amount = Double.parseDouble(cleaned);
        if (amount <= 0) {
            throw new InvalidAmountException("The amount must be greater than zero.");
        }
        return amount;
    }

    /** True when the PIN is exactly four digits (null and letters fail). */
    public static boolean isValidPin(String pin) {
        return pin != null && PIN_PATTERN.matcher(pin).matches();
    }

    /** True when the account number is 4 to 10 digits. */
    public static boolean isValidAccountNumber(String accountNumber) {
        return accountNumber != null && ACCOUNT_PATTERN.matcher(accountNumber).matches();
    }
}
```

D.4 WithdrawFrame.java (Kgonyane Makgoale)

```
/*
 * File:        WithdrawFrame.java
 * Author:      Kgonyane Makgoale
 * Role:        Member 6, Withdraw screen
 * Group:       Group 2, SDN150S (Software Design 1), CPUT
 * Description: Lets the customer withdraw cash. The amount is validated,
 *              then the ATM applies the cash rules (whole notes, balance,
 *              daily limit). Success and every kind of error are shown with
 *              a clear message.
 */
package atm;

import java.awt.BorderLayout;
import java.awt.Dimension;
import javax.swing.Box;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class WithdrawFrame extends JFrame {

    private static final long serialVersionUID = 1L;

    // ------------------------------------------------------------------
    // Fields
    // ------------------------------------------------------------------
    private final ATM atm;
    private JTextField txtAmount;      // the amount to withdraw
    private JButton btnConfirm;
    private JButton btnBack;
    private JLabel lblLimit;           // how much may still be withdrawn today
    private JLabel lblMessage;         // success or error feedback

    /** Builds and shows the withdraw screen. */
    public WithdrawFrame(ATM atm) {
        this.atm = atm;

        UIStyle.styleFrame(this, "Withdraw");

        JPanel pnlMain = UIStyle.createScreenPanel();
        pnlMain.add(UIStyle.createHeader("Withdraw",
                "Notes only: enter a multiple of R" + (int) AppConstants.NOTE_DENOMINATION),
                BorderLayout.NORTH);
        pnlMain.add(buildForm(), BorderLayout.CENTER);
        pnlMain.add(buildFooter(), BorderLayout.SOUTH);
        add(pnlMain);

        // Button logic
        btnConfirm.addActionListener(e -> handleConfirm());
        btnBack.addActionListener(e -> goBack());
        getRootPane().setDefaultButton(btnConfirm);      // Enter confirms

        refreshLimit();
        setVisible(true);
        txtAmount.requestFocusInWindow();
    }

    // ------------------------------------------------------------------
    // Layout
    // ------------------------------------------------------------------

    private JPanel buildForm() {
        txtAmount = UIStyle.createTextField();
        lblLimit = UIStyle.createHintLabel(" ");

        JPanel card = UIStyle.createCardPanel();
        card.add(UIStyle.createLabel("Amount (R)"));
        card.add(Box.createRigidArea(new Dimension(0, 6)));
        card.add(txtAmount);
        card.add(Box.createRigidArea(new Dimension(0, 12)));
        card.add(UIStyle.createQuickAmountPanel(txtAmount, AppConstants.getQuickAmounts()));
        card.add(Box.createRigidArea(new Dimension(0, 12)));
        card.add(lblLimit);

        JPanel holder = UIStyle.createColumn();
        holder.add(card);
        return holder;
    }

    private JPanel buildFooter() {
        lblMessage = UIStyle.createMessageLabel();
```

```
        btnConfirm = UIStyle.createButton("Confirm withdrawal");
        btnBack = UIStyle.createSecondaryButton("Back to menu");

        JPanel footer = UIStyle.createColumn();
        footer.add(lblMessage);
        footer.add(Box.createRigidArea(new Dimension(0, 6)));
        footer.add(btnConfirm);
        footer.add(Box.createRigidArea(new Dimension(0, 10)));
        footer.add(btnBack);
        return footer;
    }

    // ------------------------------------------------------------------
    // Button logic
    // ------------------------------------------------------------------

    /** Validates the amount, asks the ATM to withdraw and reports the outcome. */
    private void handleConfirm() {
        try {
            double amount = InputValidator.parseAmount(txtAmount.getText());
            atm.withdraw(amount);

            UIStyle.showSuccess(lblMessage, "Withdrawal successful: " + UIStyle.formatMoney(amount)
                    + "<br>New balance: " + UIStyle.formatMoney(atm.getBalance()));
            txtAmount.setText("");

        } catch (InvalidAmountException ex) {
            // Not a valid amount, not whole notes, or over the daily limit
            UIStyle.showError(lblMessage, ex.getMessage());

        } catch (InsufficientFundsException ex) {
            UIStyle.showError(lblMessage, "Insufficient funds.<br>Available balance: "
                    + UIStyle.formatMoney(ex.getAvailable()));

        } finally {
            // Runs after every attempt, so the limit shown is always up to date
            refreshLimit();
            txtAmount.requestFocusInWindow();
        }
    }

    /** Shows how much more may be withdrawn today. */
    private void refreshLimit() {
        lblLimit.setText("Still available to withdraw today: "
                + UIStyle.formatMoney(atm.getRemainingDailyLimit()));
    }

    /** Returns to the main menu. */
    private void goBack() {
        dispose();
        new MainMenuFrame(atm);
    }
}
```